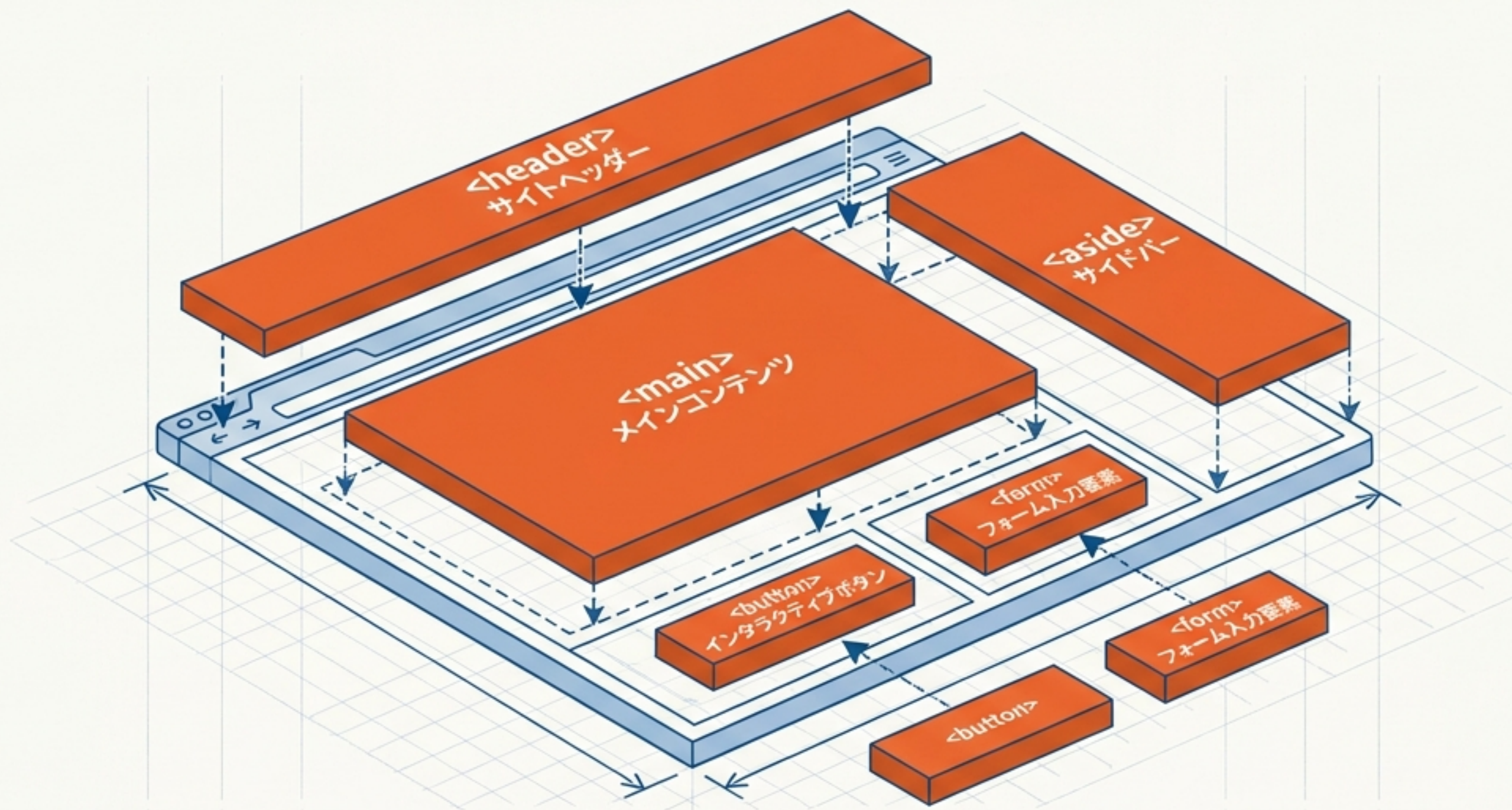


HTML5プロフェッショナル認定試験 レベル1 対策

セマンティック要素とインタラクティブ要素の完全攻略



構造設計図：HTML5要素の3つの柱

意味論と構造

<figure>

<figcaption>

<hr>

<menu>

UI・インタラクション

<details>

<summary>

<dialog>

外部リソースとスクリプト

<iframe>

<template>

<script>

<slot>

<noscript>

<canvas>



試験対策ポイント: HTML5の目的は「見た目」ではなく「意味」と「機能」をブラウザに正確に伝えること。

独立コンテンツの定義：figure と figcaption

✓ 正しい配置（最初か最後の子要素）

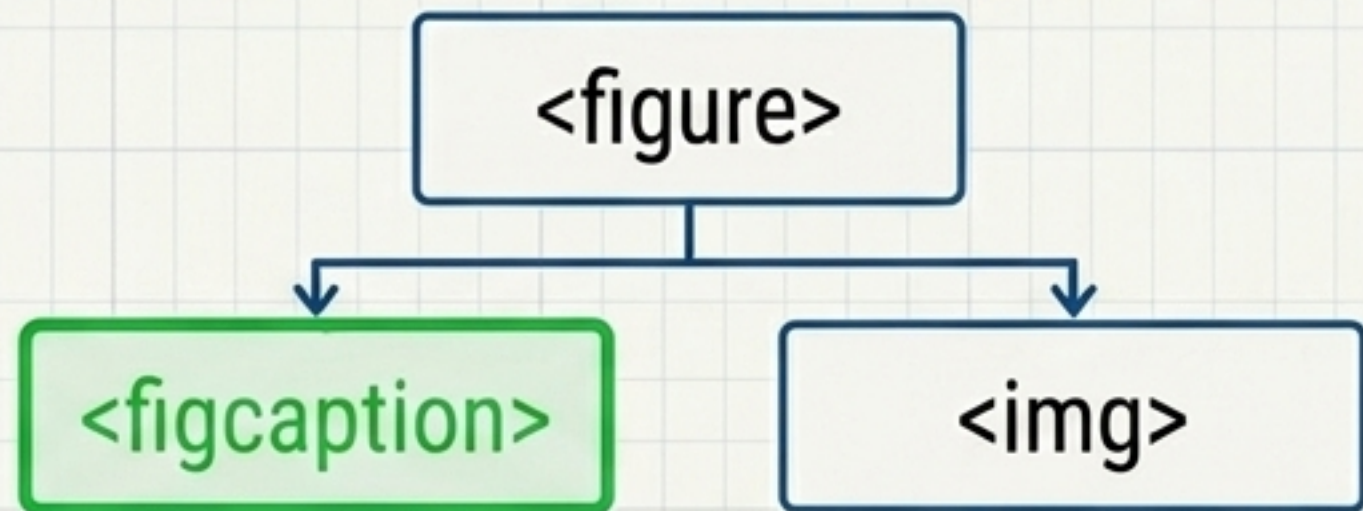
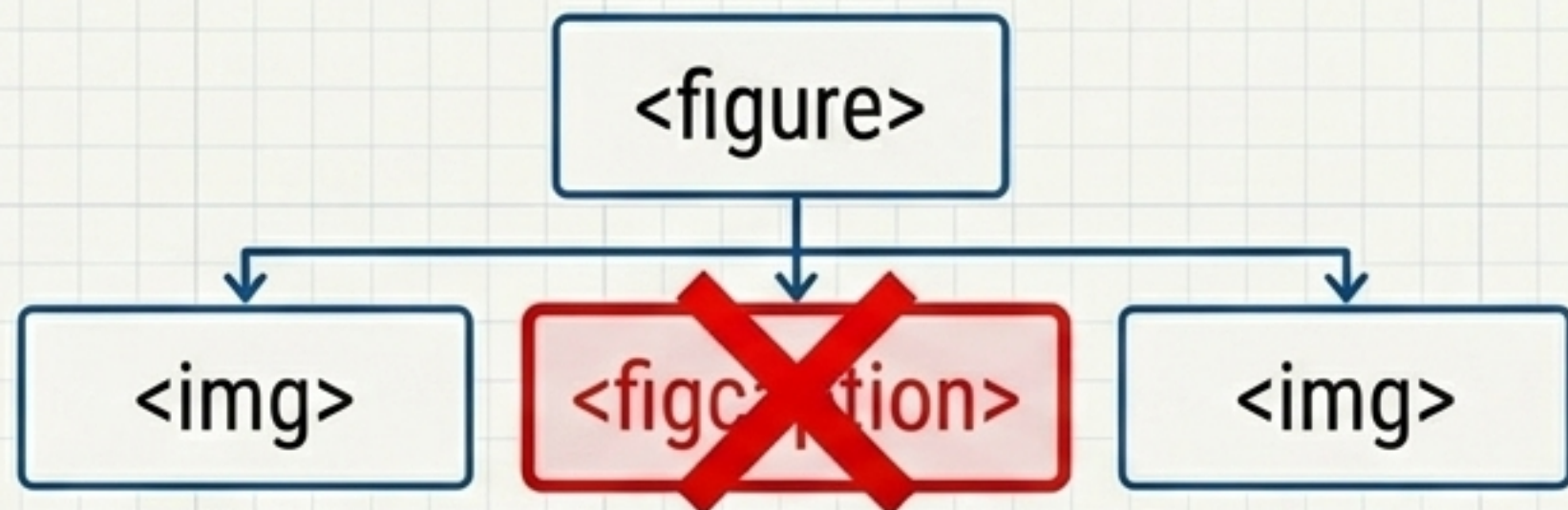


figure: 本文から参照される独立したコンテンツ。削除しても本文の意味が成立することが条件。

figcaption: figureのキャプション。必ず最初か最後の子要素として配置する。

✗ 誤った配置（中間に配置）

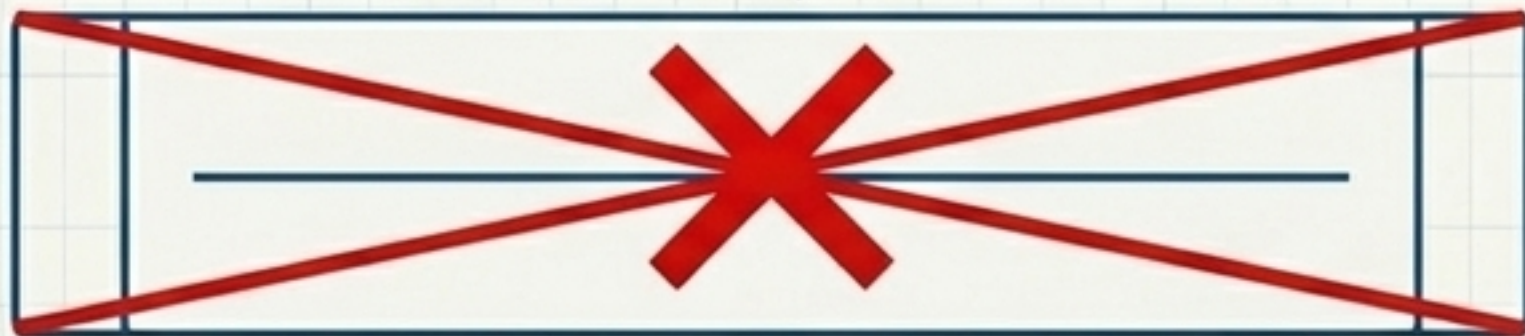


※ figcaptionを兄弟要素の間に挟むことは文法エラー

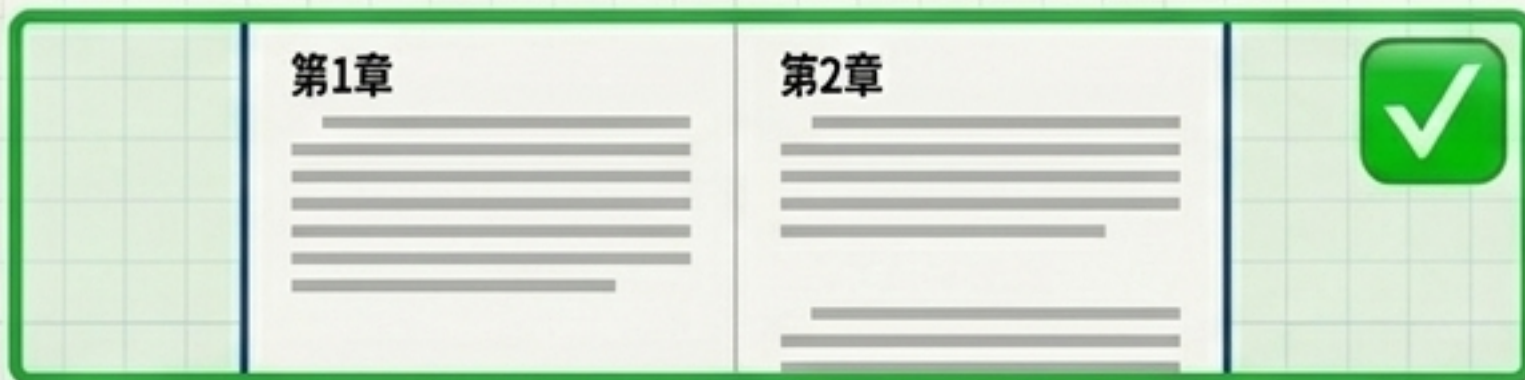
視覚から意味論への転換：hr と menu

<hr>

視覚的な水平線 (Before HTML5)



意味的な区切り (HTML5)



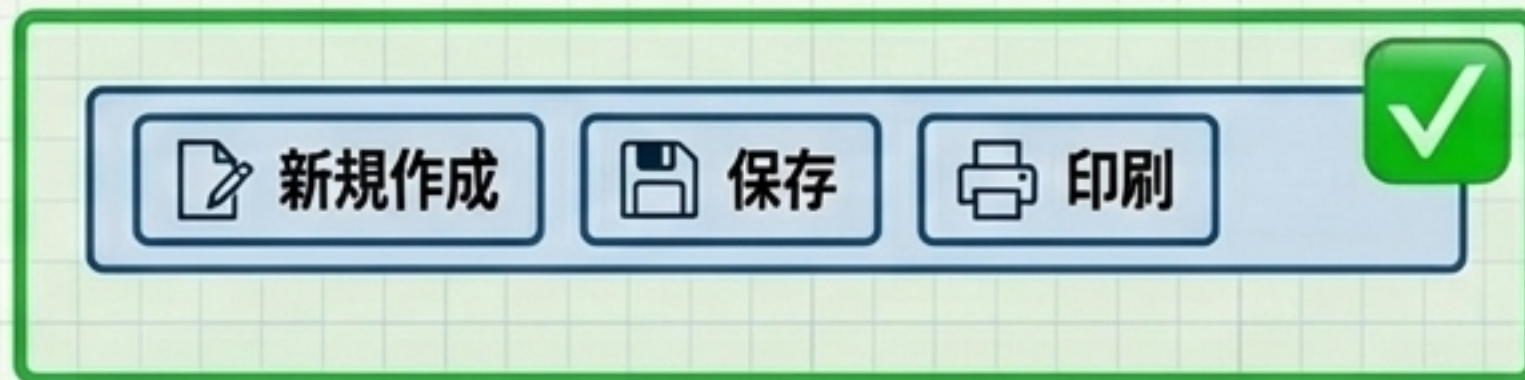
「水平線を引く」見た目のタグではなく、
「段落レベルのテーマの区切り」を表す空要素。
見た目はCSSで制御する。

<menu>

単なるリスト (Before HTML5)



インタラクティブなツールバー (HTML5)



単なるリスト (ul) ではなく、インタラクティブな
「コマンドやボタンのリスト」を表す。

ネイティブの折りたたみUI：details と summary

State 1 - Closed

▶ 詳細

State 2 - Open

▼ 詳細

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco labore.

```
<details open>
  <summary>詳細</summary>
  <p>ここに詳細コンテンツ</p>
</details>
```

open属性を付けると初期状態で展開される。

開閉時にJSの toggle イベントが発火する。

プロパティはBoolean値 (details.open = true/false)。

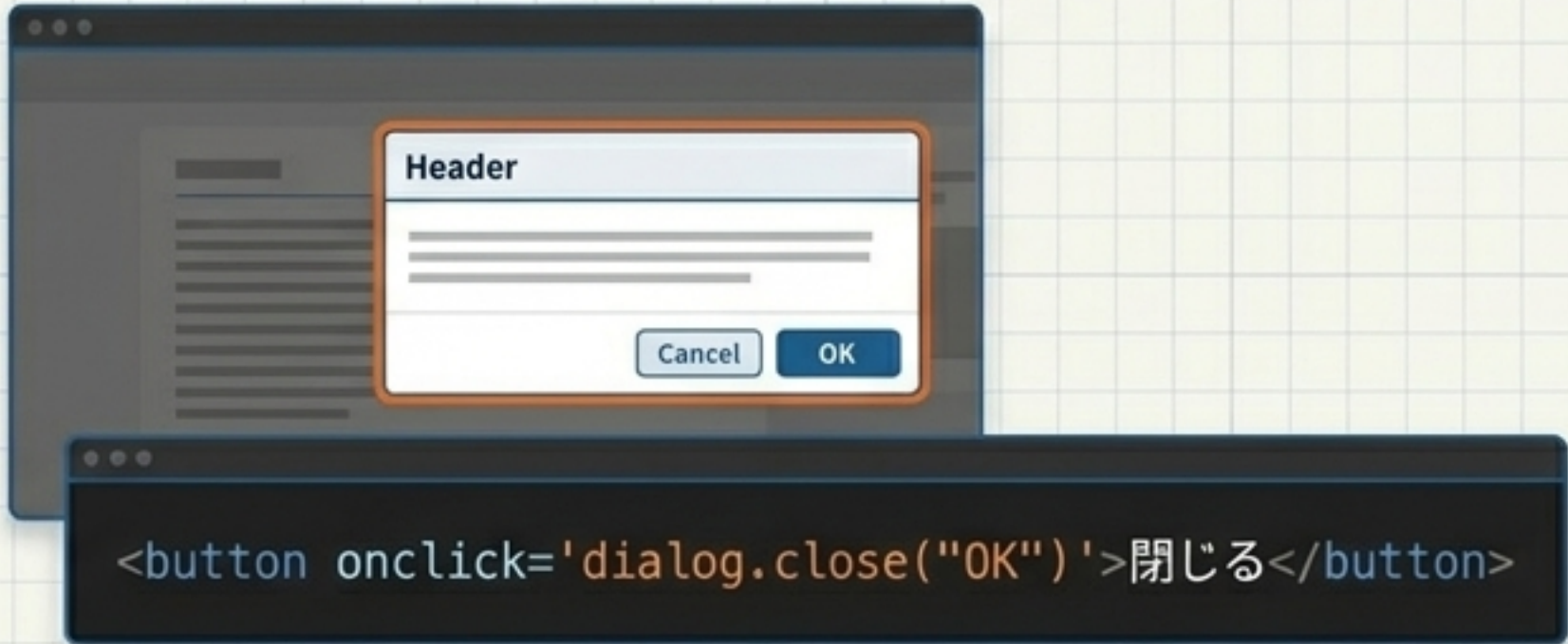
モーダルと通常表示の切り分け：dialog要素

| | 通常ダイアログ (Normal) | モーダルダイアログ (Modal) |
|----------|------------------|-----------------------|
| 呼び出しメソッド | .show() | ".showModal()" |
| 背景操作 | 操作可能 (Allowed) | 操作不可 (Blocked) |
| 背景スタイル | ::backdrop 無効 | ::backdrop 有効 |

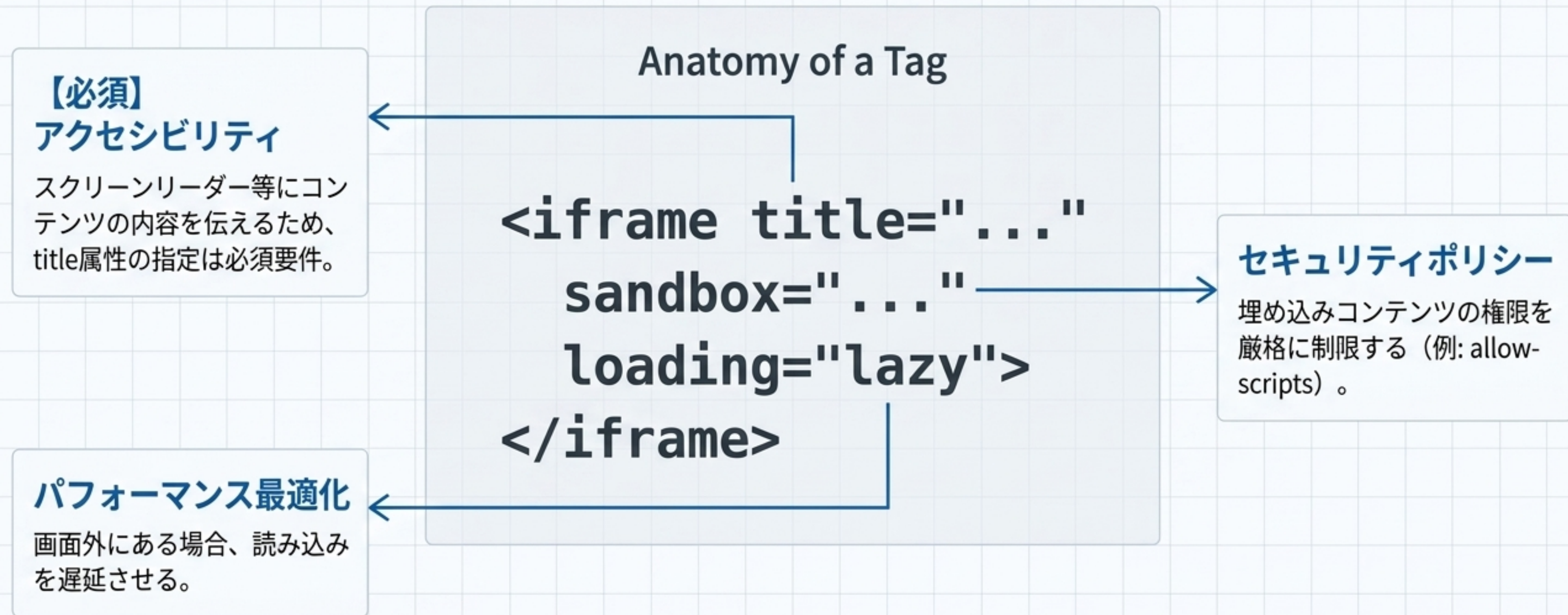
通常ダイアログ



モーダルダイアログ



外部コンテンツの安全な埋め込み：iframe



⚠ 注意: セキュリティポリシー（X-Frame-Options等）により、iframe内で表示がブロックされる場合がある。

スクリプト実行のタイミング制御：defer と async

Blue Bar = HTML解析 (Parsing)
Yellow Bar = JSダウンロード
Red Bar = JS実行 (Execution)

なし
(Default)



HTML解析をブロック。すぐにダウンロードして実行。

defer



【順序保証あり】 HTML解析完了後に実行。DOM操作に最適。

async



【順序保証なし】 ダウンロード完了次第すぐ実行。独立した解析スクリプト等に最適。



補足: `<noscript>`要素はJavaScriptが無効な環境向けの代替コンテンツ（JS有効時は非表示）。

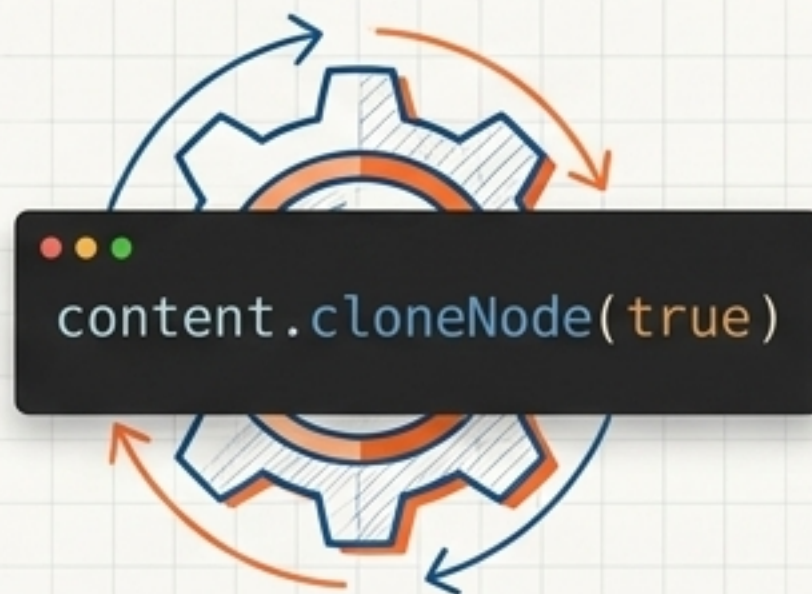
非レンダリングの雛形：template と slot

Step 1: 雛形（非表示）

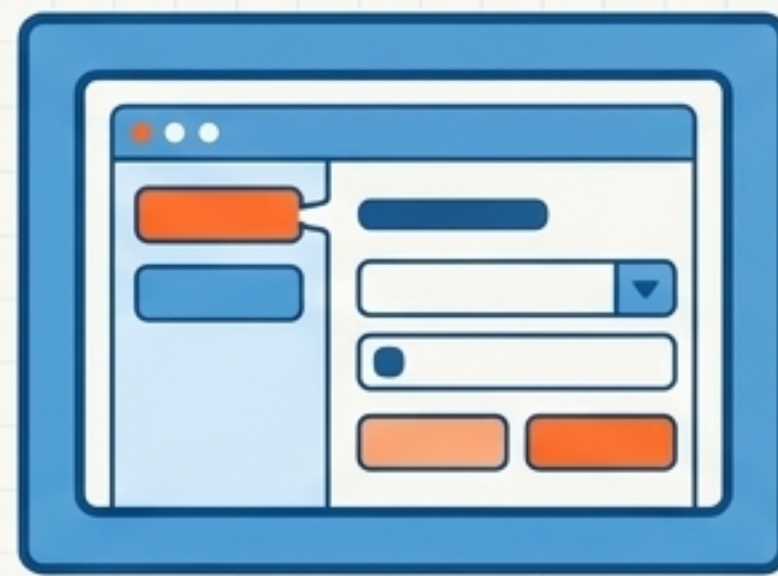


ページ読み込み時にはレンダリングされない。

Step 2: 複製（cloneNode）



Step 3: DOM追加



実体化して画面に表示される。

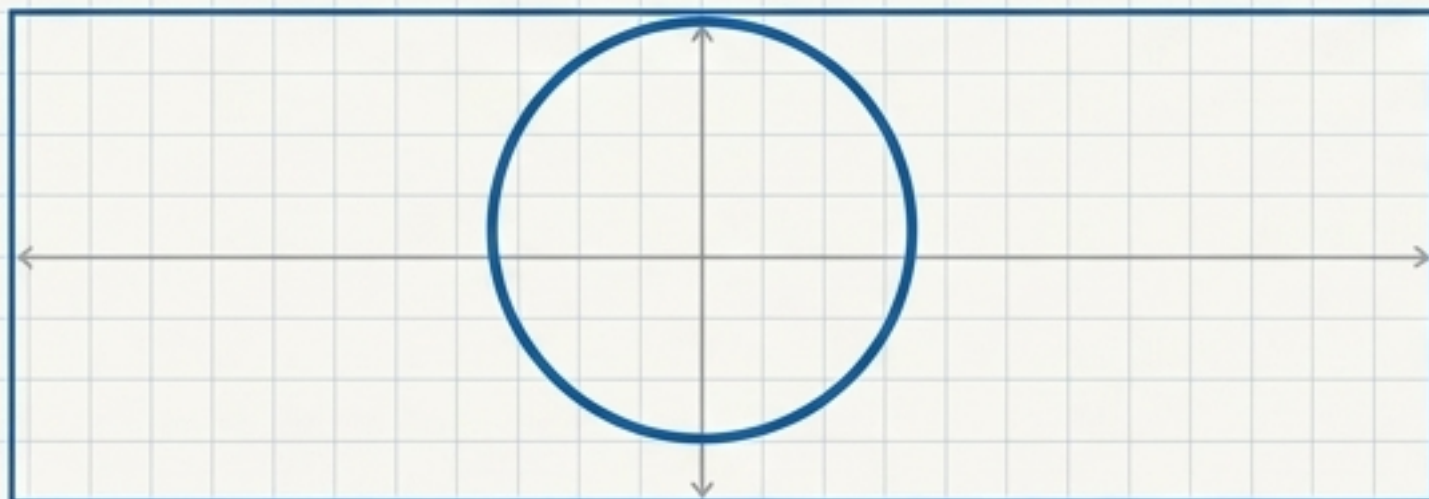
関連概念: `<slot>`

Web Components（Shadow DOM）におけるコンテンツ挿入口。name属性で名前付きスロットを定義可能。

グラフィックコンテキストの罫：canvas要素

✅ HTML属性による解像度指定

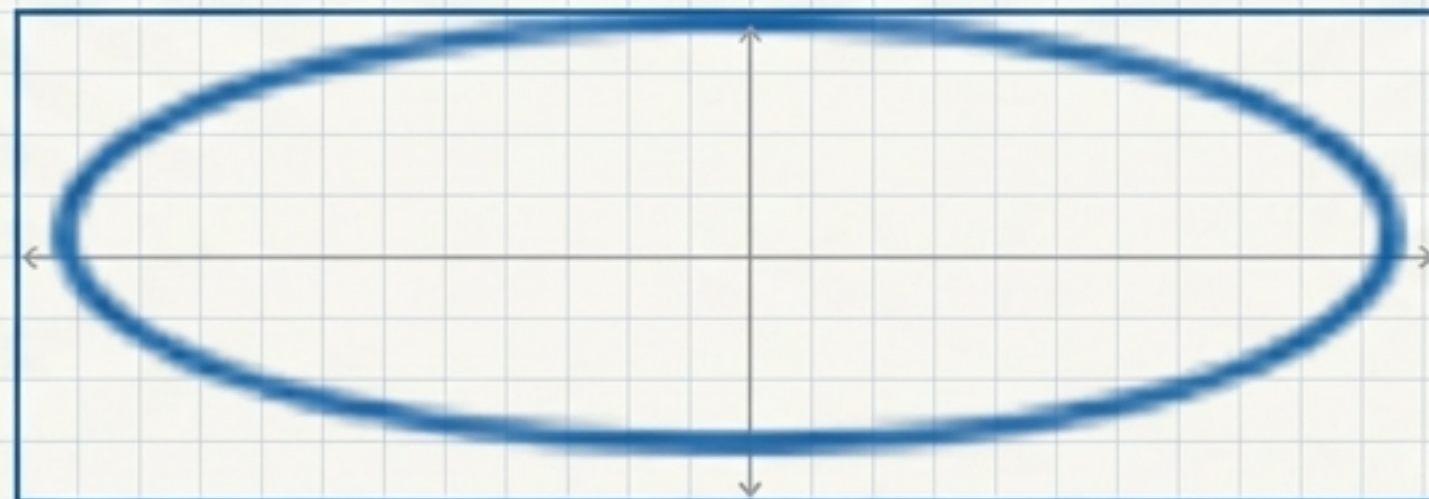
```
<>
<canvas width="500" height="160">
</>
```



描画解像度を指定（CSSとは別物）。鮮明な描画。

❌ CSSによるサイズ変更の罫

```
canvas {
  width: 500px;
  height: 160px;
}
```



⚠️ CSSでサイズを変えるとピクセルが引き伸ばされ歪む。

- getContext('2d')で2D描画コンテキストを取得してJavaScriptで描画。
- canvasはピクセル（ビットマップ）ベース。拡大すると粗くなる（ベクターベースのSVGとは異なる）。
- 非対応環境向けにタグの間に代替テキストを記述可能。

Level 1 直前対策：マスターシートシート

セマンティック (Semantics)

`<figure>` : 独立コンテンツのラッパー。

`<figcaption>` : 最初か最後の子要素に配置するキャプション。

`<menu>` : コマンドやボタンのリスト (ツールバー等)。

`<hr>` : 段落レベルのテーマの区切り (見た目の線ではない)。

インタラクティブ (Interactive)

`<details>` : クリックで開閉する折りたたみUI。open属性で初期展開。

`<summary>` : details 内の見出し (クリック対象)。

`<dialog>` : ネイティブモーダル。showModal() と ::backdrop。

メタデータ&スクリプト (Scripts)

`<script>` : defer (順序保証あり) / async (順序保証なし)。

`<noscript>` : JS無効環境向けの代替コンテンツ。

`<template>` : 非レンダリングの雛形。cloneNode() で複製。

`<slot>` : Web Componentsのコンテンツ挿入口。

埋め込み (Embeds)

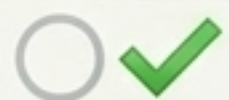
`<iframe>` : 外部ページの埋め込み。title属性必須。

`<canvas>` : JSで描画。width/height属性はCSSサイズとは別物。

模擬試験：概念チェック（Part 1）

Q1（正誤問題）

HTML5において <hr> 要素は「水平線を引く」という見た目上の意味で定義されている。



Q2（選択問題）

<script> 要素の defer 属性に関する説明として正しいものはどれか？

- ☐ ダウンロード完了次第すぐ実行される。
- ☒ HTML解析完了後に記述順で実行される（順序保証あり）。
- ☐ HTML解析をブロックして実行される。

Q3（複数選択問題） 正しい記述をすべて選べ。

- | | |
|---|---|
| ☐ <figcaption>は最初か最後の子要素。 | ☐ dialog.showModal()で::backdropが有効になる。 |
| ☐ <details open>で初期展開される。 | ☐ <iframe>のtitle属性はアクセシビリティのために必須である。 |
| ☐ <canvas>のwidth/heightはCSSのサイズ指定と同じである。 | |

模擬試験：コード穴埋め (Part 2)



1. 画像とキャプションをまとめる要素

```
<[_____]>  
  <img src='photo.jpg' alt='写真'>  
  <figcaption>図1：写真の説明</figcaption>  
</[_____]>
```

2. 最初から展開された状態のdetails

```
<details [_____]>  
  <summary>詳細</summary>  
</details>
```

3. dialogをモーダル表示するメソッド

```
document.getElementById('myDialog').[_____]();
```

4. templateの中身を複製するメソッド

```
const clone = tpl.content.[_____](true);
```

5. canvasで2Dコンテキストを取得

```
const ctx = canvas.getContext([_____]);
```

6. HTML解析後・順序保証ありでスクリプトを実行

```
<script src='app.js' [_____]></script>
```


解答と解説

Part 1: 概念チェック

Q1: **×** (偽)

見た目ではなく「段落レベルのテーマの区切り」という【意味論的】な要素。

Q2: HTML解析完了後に記述順で実行される。
defer属性は順序保証あり。DOM操作に安全。

Q3: 1, 2, 4, 5 が正しい。

(3は間違い: canvasのwidth/heightはCSSサイズとは別物であり、CSSで変更すると歪む)。

Part 2: コード穴埋め

A: figure

B: open

C: showModal

D: cloneNode

E: '2d'

F: defer

 **試験対策の急所:** Level 1試験では、仕様の概念理解だけでなく、属性のスペル (defer, open) やメソッドの正確なキャメルケース表記 (cloneNode, showModal) を直接問われる問題が頻出します。正確なタイピングレベルでの記憶が必要です。

合格へのネクストステップ

- ✓ figure / figcaption
- ✓ details / summary
- ✓ menu
- ✓ iframe
- ✓ hr
- ✓ script / noscript
- ✓ dialog
- ✓ template / slot
- ✓ canvas

「仕様書を丸暗記するのではなく、実際にコードを書き、ブラウザでの挙動（DOMの構築、イベントの発火、描画タイミング）を確かめることが最短の合格ルートです。」

→ 実際の開発環境で template の複製や dialog のモーダル表示を実装してみましょう。
HTML5プロフェッショナル認定試験での健闘を祈ります！